Second Thought (再想想) – Security Self-Assessment Report

Oct 20, 2026 · @Alexander

Scope: the submitted repository running against synthetic demo data, user-exported Alipay and WeChat Pay bills and the Plaid Sandbox. Result of our assessment: the language model cannot move money on its own, automatic actions are limited to rules the user set while calm, and every decision is logged in a tamper-evident chain. The main open risk is that holds act only in the app and at browser checkouts until a bank partner provides card controls.

1. Principles

1. **The model talks, the code calculates.** All figures come from deterministic Python. The language model explains results and chooses tools; it never produces a number the user acts on.
2. **The model never moves money on its own.** Its strongest move is a proposal that waits for a person to tap Confirm.
3. **Rules are set while calm.** Automatic (T3) actions run only from the user's calm-state plan, through a deterministic rules engine, within caps.
4. **Asymmetric friction.** Changes that add protection apply immediately; changes that remove protection or give the agent more power wait 24 hours.
5. **No health data.** Users choose spending patterns, never diagnoses, so the system holds no medical information.

2. Permission-tier implementation

Every action, whether it comes from the user interface, the agent or the rules engine, goes through `Gate.call()` in `second_thought/policy/gate.py`. Unknown tools are denied; T3 tools are denied to any caller except the rules engine; each tool's safety check runs before execution; T2 tools become pending proposals unless the user confirmed them; every outcome is logged.

Tier	Tools	Enforcement in code	Tests
T0 Observe	`get_safe_to_spend`, `get_bill_check`, `get_return_windows`, `get_money_mirror`, `get_recent_transactions`, `get_plan`, `get_parking_lot`	Read-only functions in `services.py`, logged by the gate	`test_every_call_is_logged`
T1 Advise	`pause_check`	No side effects beyond the risk timestamp; calm-state rules are evaluated afterwards by the rules engine	`test_pause_flow_night_hold_and_auto_save`
T2 Confirm	`park_item`, `save_to_pot`, `draft_cancellation`, `draft_return`, `set_category_cap`, `request_rule_change`, `notify_ally`; UI only: `record_decision`, `resolve_parked`, `log_mood`, `cancel_pending_change`, `remove_ally_now`	`Gate.call()` stores a pending action unless the actor is the user and the call is confirmed; only `Gate.confirm()` executes it; the agent cannot set the confirmation flag	`test_agent_can_only_propose_t2`, `test_agent_cannot_confirm_itself`, `test_bills_prepare_drafts_not_sent`
T3 Bounded	`auto_hold`, `auto_save`, `auto_ally_alert`	Denied for every actor except `rules`; called only by `policy/rules_engine.py` from plan rules; checks: essentials never held, daily auto-save cap, ally alerts only when switched on	`test_t3_reserved_for_rules_engine`, `test_essentials_never_held`, `test_daily_auto_save_cap`
Never	Pay third parties, borrow or use BNPL, trade, change credentials, read data the user did not import	No such tools exist; unknown tool names are denied and logged; the offline agent answers transfer requests with a refusal	`test_unknown_tool_denied_and_logged`, `test_transfer_requests_refused`

Browser extension (Checkout Pause)

The extension adds no new powers. Its local API (`second_thought/extension_api.py`) maps every request onto the same gate:

Request	Tool and tier	Who decides
`POST /v1/checkout`	`pause_check`, T1; calm-state rules may then run `auto_hold` and `auto_save` (T3, rules engine only)	Nobody: it only scores and informs
Park it	`park_item`, T2	The user's own click in the overlay
Don't buy, Buy anyway	`record_decision` or `resolve_parked`, T2	The user's own click; Buy anyway only after a 3 to 20 second countdown

These calls are logged with actor `user` and the note "via browser extension". No other tool is reachable from the extension. Tests: `test_extension_api.py` (9) and `test_extension_detect.py` (5).

Rule changes and cooldown

`policy/plan.py` classifies each requested change. `classify()` labels it safer or riskier, `request_change()` applies safer changes and queues riskier ones for 24 hours, and `apply_due()` applies them once the cooldown has passed. Queued changes can be cancelled at any time. The trusted ally can be removed immediately through the "I feel unsafe" exit, so the role cannot be used for financial control.

Change	Class	Effective
Switch a protection on, lower a threshold or cap, block a category, add an ally	Safer	Immediately
Switch a protection off, raise a threshold or cap, unblock a category	Riskier	After 24 hours
Higher auto-save share or daily cap (more agent power)	Riskier	After 24 hours
Remove the ally	Riskier	After 24 hours, or immediately via the unsafe exit

Tests: `test_asymmetric_friction`, `test_more_agent_power_waits`, `test_ally_removal_waits_unless_unsafe`, `test_loosening_needs_confirmation_then_waits`.

3. Further controls

Control	Implementation	Tests
Tamper-evident audit log	`policy/audit_log.py`: append-only JSON lines; each entry stores the SHA-256 of the previous one; `verify()` returns the first broken entry. Proposals, confirmations, denials and rule outcomes are logged; chat text is not stored, only a safety flag and its length	`test_audit_chain_detects_tampering`
Prompt injection	Transaction text is treated as data. The offline agent reads only what the user typed. The LLM agent is told never to follow instructions inside tool results and can at most propose T2 actions; T3 is unreachable for it	`test_prompt_injection_in_transaction_text_does_nothing`
Redaction before model calls	`agent/safety.py` `Redactor`: names, emails and phone numbers masked; long order and account numbers replaced by reversible tokens so the model can still refer to a transaction	`test_redactor_round_trip`

Control	Implementation	Tests
Consent	Separate consent for processing financial data at onboarding; separate opt-in for sending redacted data to a model; consent changes logged	Onboarding form, audit log
Data residency	State stored per region (`.st_state/cn`, `.st_state/eu`); CN users only use CN-hosted model providers	`test_cn_users_only_use_cn_hosted_models`
Encryption at rest	With `ST_ENCRYPTION_KEY`, the state file and every imported bill export are encrypted with Fernet (AES-128-CBC with HMAC-SHA256)	Manual check in the build log
Secrets	Plaid and model keys only from environment variables; the Plaid request log stores endpoint, status and request id, never keys or tokens	`test_plaid_mapping_sign_and_categories`
Crisis protocol	Strong signals in English, Chinese and German stop the finance flow and show local help (12356 in China, TelefonSeelsorge in Germany, emergency numbers); softer distress adds a help line	`test_crisis_stops_finance_flow`

Control	Implementation	Tests
Labelling	Model replies are labelled "AI-generated", offline replies "Automated reply"	UI
Scope	Budgeting and behaviour only; no diagnosis, no treatment claims, no investment advice, no ads, no credit products	System prompt, UI disclaimer

4. Known-risk list

#	Risk	Impact	Current mitigation	Open item
1	Holds act in the app and, with the extension, at checkouts on listed shops; a payment in another app, a mini program or with the extension switched off still goes through	High	Holds park the purchase and inform the user; the extension stops the checkout click and asks twice; clearly labelled as sandbox	Card controls through a bank partner, or a capped e-CNY sub-wallet
2	The system holds an essential purchase (medicine, food, transport)	High	Essentials are never held (`_check_hold`); every hold can be overridden in one tap with an optional reason	Category detection can be wrong (see 9)
3	The model states a wrong number	High	Numbers come only from tool results; offline replies use code templates	Check that every number in a model reply appears in a tool result
4	Prompt injection through merchant names or order notes	High	Transaction text is data only; T3 is unreachable for the model; T2 needs a tap	Red-team the LLM mode with more injection samples

#	Risk	Impact	Current mitigation	Open item
5	A user in a hot state removes protections	Medium	24-hour cooldown for riskier changes, cancellable at any time	Users can still delete the app; we accept that the goal is a pause, not control
6	The trusted ally misuses the role (financial control)	High	The ally only gets a short heads-up, never amounts or transactions; removal is possible, immediately via the unsafe exit	Ally messages are only queued in the sandbox, not sent
7	A crisis disclosure is missed	High	Phrase lists in English, Chinese and German; finance flow stops on a match	Reviewed classifier and helpline data per country
8	Data breach	High	Optional encryption at rest; no bank passwords (bill exports or OAuth tokens only); regional stores	Audit log is not yet encrypted (amounts and item titles, no chat text)
9	Wrong category or false impulse flag	Low	Every point of the score is shown as a reason; the user can always override	No in-app category correction yet
10	The model provider sees personal data	Medium	Opt-in, redaction, CN users only on CN-hosted models	A production launch in China would use a model with completed filing
11	The app is seen as medical or financial advice	Medium	Disclaimers in the app and system prompt; patterns instead of diagnoses; no investment advice	Legal review before launch
12	Static FX rates	Low	Labelled as demo rates	Daily PBoC and ECB reference rates

#	Risk	Impact	Current mitigation	Open item
13	The offline agent misreads a request	Low	Same gate as the LLM agent, so a misread can only show data or prepare a proposal	More intents, or LLM mode
14	A web page or another local program uses the extension's local API	High	Binds to 127.0.0.1 only; pairing code on every call, checked in constant time, lockout after 10 wrong codes; Host and Origin checks, no CORS headers; 16 KB body limit; the API reaches one T1 tool and the user's own T2 decisions only	Pairing code sits in extension storage and a local file; move it to the OS keychain
15	The extension misses a checkout button or reads the wrong price	Medium	Labels in Chinese, English and German; total line, structured data and price elements checked in that order, crossed-out prices skipped; the user sees the amount and can edit it	Shop-specific selectors with partners; more languages
16	The extension reads shopping pages (privacy)	Medium	Runs only on listed shop domains; sends only amount, title, shop name and matched signal words, and only to the user's own computer; no history, cookies or page content	Store review (Chrome Web Store, Edge Add-ons) and a privacy notice before release

5. How to verify

```
pytest -q                                                    # 51 tests,
including every test named above
python scripts/run_demo.py                                   # writes
evidence/ with transcript and audit log
python scripts/verify_audit_log.py evidence/audit_log.jsonl # recomputes the
hash chain
python scripts/extension_e2e.py                              # real Chromium:
extension, local API, demo shop
```

Evidence files in the repository folder `evidence/`: `demo_transcript.md` (each request with the gate's decision, including refused actions and the crisis protocol), `audit_log.jsonl` (35 hash-chained entries), `sandbox_operations.json` (sandbox transfers, drafts, parking lot, queued rule changes), `plaid_requests.jsonl`, `summary.json` and `demo_video.mp4`.

For the extension, `evidence/extension/` holds screenshots of both warning steps in English and Chinese, the settings page, the popup and offline mode, and `e2e_results.txt`, which lists the score, the reasons, the money-left figures and the three audit entries written via the extension.